

Python Jupyter Notebook Tutorial

By Johannes Simatupang, M.Kom., COBIT5 (Feb 2024)

COMP6577 - Machine Learning >>> Bina Nusantara University

```
In [1]: # Python 3.xx  
print('Hello World!!!')
```

Hello World!!!

```
In [2]: # Python 2.xx  
print "Hello World!!"
```

File "C:\Users\Johannez\AppData\Local\Temp\ipykernel_11928\2404951875.py", line 2

print "Hello World!!"

^

SyntaxError: Missing parentheses in call to 'print'. Did you mean print("Hello World!!")?

```
In [3]: # this is a comment  
print("Welcome to Python Tutorial") #comment after a statement.
```

Welcome to Python Tutorial

Markdown

Markdown is a lightweight, easy to learn markup language for formatting plain text. Its syntax has a one-to-one correspondance with HTML tags, so some prior knowledge here would be helpful but is definitely not a prerequisite. Remember that this article was written in a Jupyter notebook, so all of the narrative text and images you have seen so far was achieved in Markdown. Let's cover the basics with a quick example.

Here's how to format Markdown cells in Jupyter notebooks:

This is a Level 1 heading

This is a Level 2 heading

This is a level 1 heading

This is a level 2 heading

This is some plain text that forms a paragraph. Add emphasis via **bold** and bold, or *italic* and italic. Paragraphs must be separated by an empty line.

- Sometimes we want to include lists.
- Which can be indented.

1. Lists can also be numbered.

2. For ordered lists. It is possible to include hyperlinks (<https://www.example.com>) Inline code uses single backticks: `foo()` , and code blocks use triple backticks:

```
bar()
```

Or can be indented by 4 spaces: `foo()` And finally, adding images is easy:

Python Data Types

Data types are the classification or categorization of data items. Python supports the following built-in data types.

1. Scalar Types (int, float, complex, bool, none)
2. Sequence Type (string, list, tuple)
3. Mapping Type (Dictionary)
4. Set Types (Set)
5. Mutable and Immutable Types Numbers, strings, and Tuples are immutable, which means their contents can't be altered after creation. On the other hand, items in a List or Dictionary object can be modified.

```
In [4]: #String
str1='Welcome to "Python Tutorial" on Tutorial'
print(str1)
str2="Welcome to 'Python Tutorial' on Tutorial"
print(str2)
```

```
Welcome to "Python Tutorial" on Tutorial
Welcome to 'Python Tutorial' on Tutorial
```

```
In [5]: #List
mylist=[] # empty list
print(mylist)
names=["Jeff", "Bill", "Steve", "Mohan"] # string list
print(names)
item=[1, "Jeff", "Computer", 75.50, True] # list with heterogeneous data
print(item)
```

```
[]
['Jeff', 'Bill', 'Steve', 'Mohan']
[1, 'Jeff', 'Computer', 75.5, True]
```

```
In [6]: #tuple
tpl=() # empty tuple
print(tpl)
names = ('Jeff', 'Bill', 'Steve', 'Yash') # string tuple
print(names)
nums = (1, 2, 3, 4, 5) # int tuple
print(nums)
employee=(1, 'Steve', True, 25, 12000) # heterogeneous data tuple
print(employee)
```

```
()
('Jeff', 'Bill', 'Steve', 'Yash')
(1, 2, 3, 4, 5)
(1, 'Steve', True, 25, 12000)
```

```
In [7]: #Set Data Type & Operations
s1={1,2,3,4,5}
s2={4,5,6,7,8}
s1.union(s2)
```

```
Out[7]: {1, 2, 3, 4, 5, 6, 7, 8}
```

```
In [8]: s1.intersection(s2)
```

Out[8]: {4, 5}

```
In [9]: #if statement
price = 50
quantity = 5
if price*quantity < 500: #conditional if statement
    print("price*quantity is less than 500")
    print("price = ", price)
    print("quantity = ", quantity)
```

price*quantity is less than 500
price = 50
quantity = 5

```
In [10]: #if else Condition
price = 50
if price >= 100:
    print("price is greater than 100")
else:
    print("price is less than 100")
```

price is less than 100

```
In [11]: # Function in Python
def say_hello(recipient):
    return 'Hello, {}'.format(recipient)
say_hello('Tim')
```

Out[11]: 'Hello, Tim!'

```
In [12]: #using numpy library
import numpy as np
def square(x):
    return x * x
x = np.random.randint(1, 10)
y = square(x)
print('%d squared is %d' % (x, y))
```

7 squared is 49

```
In [13]: #Looping using For
for x in range(6):
    print(x)
else:
    print("Finally finished!")
```

```
0
1
2
3
4
5
Finally finished!
```

```
In [14]: #Class in Python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
p1 = Person("John", 36)
print(p1.name)
print(p1.age)
```

```
John
36
```

```
In [15]: #inherit in python
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname
    def printname(self):
        print(self.firstname, self.lastname)
#Use the Person class to create an object, and then execute the printname method:
x = Person("John", "Doe")
x.printname()
```

```
John Doe
```

```
In [16]: #Iterate tuple
mytuple = ("apple", "banana", "cherry")
for x in mytuple:
    print(x)
```

```
apple
banana
cherry
```

End of Practise